Software Testing Document

– Chess Game –

1. Introduction and Purpose

The System Under Development (SUD) is a programmed chess application that allows a user to play against an opponent. The opponent can either be a real-life player or an Artificial Intelligence (AI) bot powered by an Application Programming Interface (API). The SUD is intended to allow players to view a fully visualized chess board and pieces and interact with them to compete in a game of chess following standard chess rules.

1.1 Acronyms, Terms and References

Acronym	Meaning
HDD	High-Level Design
LLD	Low-Level Design
SDD	Software Design Document
SUD	System Under Development
SWEBOK	Software Engineering Body of Knowledge
GUI	Graphical User Interface
API	Application Programming Interface
AI	Artificial Intelligence
FEN	Forsyth-Edwards Notation
TC	Test Case
TP	Test Procedure

Term	Definition

Forsyth-Edwards Notation	Standard Notation to describe the position of a chess game in software(see FEN reference below)
Environment Variable	Variables stored at an operating system level, outside of the source code itself (see Dagster's explanation below)

Reference	Source
FIDE Laws of Chess	https://www.fide.com/FIDE/handbook/LawsOfChess.pdf
FEN Reference	https://www.chess.com/terms/fen-chess
Pygame Documentation	https://www.pygame.org/docs/
Python 'dotenv' Package	https://pypi.org/project/python-dotenv/
Dagster's Explanation of Environment Variables in Python	https://dagster.io/blog/python-environment-variables

2 Verification Philosophy

This verification approach combines black-box and white-box testing, along with structured reviews and analysis techniques. Testing is organized into a 1:M hierarchy where each Test Case targets a behavior, and each Test Procedure represents a specific scenario.

3 Test Cases

TC1: FEN Parsing & Serialization:

Validates board ↔ FEN correctness and internal state consistency.

Black-Box Test Procedures:

- [STD_TC1_BB_TP01]: Input standard starting FEN → verify correct board
- [STD_TC1_BB_TP02]: Input mid-game FEN → verify piece placement

- [STD_TC1_BB_TP03]: Invalid FEN → expect failure/exception
- [STD_TC1_BB_TP04]: Convert board → FEN → compare to expected

White-Box Test Procedures:

- [STD_TC1_WB_TP05]: Step through fen_to_board() loop logic
- [STD_TC1_WB_TP06]: Verify digit expansion correctness
- [STD_TC1_WB_TP07]: Validate board_to_fen() compression logic
- [STD_TC1_WB_TP08]: Check state variables (turn, castling, en passant)

TC2: Legal Move Generation:

Ensures only valid chess moves are generated.

Black-Box Test Procedures:

- [STD_TC2_BB_TP01]: Pawn forward, capture, double move
- [STD_TC2_BB_TP02]: Knight movement from center and edges
- [STD_TC2_BB_TP03]: Bishop diagonal movement
- [STD_TC2_BB_TP04]: Rook straight movement
- [STD_TC2_BB_TP05]: Queen combined movement
- [STD_TC2_BB_TP06]: King movement in all directions
- [STD_TC2_BB_TP07]: Attempt illegal moves → verify exclusion

White-Box Test Procedures:

- [STD_TC2_WB_TP08]: Trace get_legal_moves() per piece type
- [STD_TC2_WB_TP09]: Validate add_move() prevents illegal states
- [STD_TC2_WB_TP10]: Confirm in_bounds() logic
- [STD_TC2_WB_TP11]: Verify check prevention logic

TC3: Special Rules:

Validates castling, en passant, and promotion.

Black-Box Test Procedures:

- [STD_TC3_BB_TP01]: Perform valid kingside castling
- [STD_TC3_BB_TP02]: Attempt invalid castling (blocked/in check)
- [STD_TC3_BB_TP03]: Execute en passant capture
- [STD_TC3_BB_TP04]: Trigger white promotion

- [STD_TC3_BB_TP05]: Trigger black promotion

White-Box Test Procedures:

- [STD_TC3_WB_TP06]: Trace castling rights updates
- [STD_TC3_WB_TP07]: Verify en passant coordinate tracking
- [STD_TC3_WB_TP08]: Check promotion state transitions
- [STD_TC3_WB_TP09]: Validate board updates after special moves

TC4: Game State Evaluation:

Ensures accurate detection of check, checkmate, and stalemate.

Black-Box Test Procedures:

- [STD_TC4_BB_TP01]: Set check scenario → verify detection
- [STD_TC4_BB_TP02]: Set checkmate position → verify winner
- [STD_TC4_BB_TP03]: Set stalemate position → verify draw
- [STD_TC4_BB_TP04]: Verify normal gameplay remains ongoing

White-Box Test Procedures:

- [STD_TC4_WB_TP05]: Trace is_in_check() logic
- [STD_TC4_WB_TP06]: Validate is_in_checkmate() loops
- [STD_TC4_WB_TP07]: Confirm stalemate conditions
- [STD_TC4_WB_TP08]: Verify game_state transitions

TC5: GUI Interaction:

Validates GUI behavior and user interaction.

Black-Box Test Procedures:

- [STD_TC5_BB_TP01]: Click piece → highlight moves
- [STD_TC5_BB_TP02]: Click invalid square → no action
- [STD_TC5_BB_TP03]: Execute valid move → update board
- [STD_TC5_BB_TP04]: Trigger promotion menu

White-Box Test Procedures:

- [STD_TC5_WB_TP05]: Trace handle_input() branching
- [STD_TC5_WB_TP06]: Verify board clicks while in menu state do nothing

- [STD_TC5_WB_TP07]: Verify that board clicks of the inactive color do nothing
- [STD_TC5_WB_TP08]: Verify that board clicks of the bot color do nothing
- [STD_TC5_WB_TP09]: Verify that a new game cannot be started while a game is ongoing
- [STD_TC5_WB_TP10]: Verify that a resign command cannot be sent when no game is active
- [STD_TC5_WB_TP11]: Verify coordinate mapping
- [STD_TC5_WB_TP12]: Check GUI state transitions

TC6: Bot Integration:

Validates communication with external Stockfish API.

Black-Box Test Procedures:

- [STD_TC6_BB_TP01]: Request move → verify valid response
- [STD_TC6_BB_TP02]: Simulate API failure → ensure graceful handling
- [STD_TC6_BB_TP03]: Verify bot move execution
- [STD_TC6_BB_TP04]: Missing API.env file --> system logs error
- [STD_TC6_BB_TP05]: Missing ENGINE MODE variable --> bot validation fails

White-Box Test Procedures:

- [STD_TC6_WB_TP06]: Trace WebSocket connection lifecycle
- [STD_TC6_WB_TP07]: Validate JSON payload structure
- [STD_TC6_WB_TP08]: Ensure exception handling paths execute
- [STD_TC6_WB_TP09]: Trace environment loading execution path
- [STD_TC6_WB_TP10]: Verify API retrieval from environment
- [STD_TC6_WB_TP11]: Confirm .env is not accessed directly in codebase

4 Review and Analysis Procedures

Code Reviews:
- Conduct peer reviews for all modules
- Verify trace tags align with requirements and design

- Check correctness, readability, and maintainability

Static Analysis:
- Inspect code for logical errors and unreachable branches
- Validate state transitions and edge conditions

Dynamic Analysis:
- Execute runtime scenarios to observe behavior
- Monitor game state transitions and API responses

Traceability Analysis:
- Ensure all test cases map to REQ and DES tags
- Confirm coverage completeness

5 Trace Tags

List the trace tags defined in the SDD and their mapping(s) to requirements in the SRD.

- [STD_TC1_BB_TP01]::[SDD_HLD_GM_03]
- [STD_TC1_BB_TP02]::[SDD_HLD_GM_03]
- [STD_TC1_BB_TP03]::[SDD_HLD_GM_05b]
- [STD_TC1_BB_TP04]::[SDD_HLD_GM_03]
- [STD_TC1_WB_TP05]::[SDD_LLD_GM_01]
- [STD_TC1_WB_TP06]::[SDD_LLD_GM_01]
- [STD_TC1_WB_TP07]::[SDD_LLD_GM_06]
- [STD_TC1_WB_TP08]::[SDD_LLD_GM_06]
- [STD_TC2_BB_TP01]::[SDD_HLD_GM_04]
- [STD_TC2_BB_TP02]::[SDD_HLD_GM_04]
- [STD_TC2_BB_TP03]::[SDD_HLD_GM_04]
- [STD_TC2_BB_TP04]::[SDD_HLD_GM_04]
- [STD_TC2_BB_TP05]::[SDD_HLD_GM_04]
- [STD_TC2_BB_TP06]::[SDD_HLD_GM_04]

- [STD_TC2_BB_TP07]::[SDD_HLD_GM_05]
- [STD_TC2_WB_TP08]::[SDD_LLD_GM_04]
- [STD_TC2_WB_TP09]::[SDD_LLD_GM_04]
- [STD_TC2_WB_TP10]::[SDD_LLD_GM_05]
- [STD_TC2_WB_TP11]::[SDD_LLD_GM_05]
- [STD_TC3_BB_TP01]::[SDD_HLD_GM_05a]
- [STD_TC3_BB_TP02]::[SDD_HLD_GM_05b]
- [STD_TC3_BB_TP03]::[SDD_HLD_GM_05a]
- [STD_TC3_BB_TP04]::[SDD_HLD_GM_05a]
- [STD_TC3_BB_TP05]::[SDD_HLD_GM_05a]
- [STD_TC3_WB_TP06]::[SDD_LLD_GM_05]
- [STD_TC3_WB_TP07]::[SDD_LLD_GM_05]
- [STD_TC3_WB_TP08]::[SDD_LLD_GM_05]
- [STD_TC3_WB_TP09]::[SDD_LLD_GM_05]
- [STD_TC4_BB_TP01]::[SDD_HLD_GM_05]
- [STD_TC4_BB_TP02]::[SDD_HLD_GM_05a]
- [STD_TC4_BB_TP03]::[SDD_HLD_GM_05a]
- [STD_TC4_BB_TP04]::[SDD_HLD_GM_05]
- [STD_TC4_WB_TP05]::[SDD_LLD_GM_05]
- [STD_TC4_WB_TP06]::[SDD_LLD_GM_05]
- [STD_TC4_WB_TP07]::[SDD_LLD_GM_05]
- [STD_TC4_WB_TP08]::[SDD_LLD_GM_05]
- [STD_TC5_BB_TP01]::[SDD_HLD_GUI_01]
- [STD_TC5_BB_TP02]::[SDD_HLD_GUI_04]
- [STD_TC5_BB_TP03]::[SDD_HLD_GUI_05]
- [STD_TC5_BB_TP04]::[SDD_HLD_GUI_02]
- [STD_TC5_WB_TP05]::[SDD_HLD_GUI_04]
- [STD_TC5_WB_TP06]::[SDD_HLD_GUI_05]
- [STD_TC5_WB_TP07]::[SDD_HLD_GUI_04]
- [STD_TC5_WB_TP08]::[SDD_HLD_GUI_01]
- [STD_TC5_WB_TP09]::[SDD_HLD_GUI_03]
- [STD_TC5_WB_TP10]::[SDD_HLD_GUI_02]
- [STD_TC5_WB_TP11]::[SDD_HLD_GUI_01]
- [STD_TC5_WB_TP12]::[SDD_HLD_GUI_01]

- [STD_TC6_BB_TP01]::[SDD_HLD_BD_03]
- [STD_TC6_BB_TP02]::[SDD_LLD_API_05]
- [STD_TC6_BB_TP03]::[SDD_HLD_BD_04]
- [STD_TC6_BB_TP04]::[SDD_LLD_API_01], [SDD_LLD_API_05]
- [STD_TC6_BB_TP05]::[SDD_LLD_API_02], [SDD_LLD_API_05]
- [STD_TC6_WB_TP06]::[SDD_LLD_API_02]
- [STD_TC6_WB_TP07]::[SDD_LLD_BD_03]
- [STD_TC6_WB_TP08]::[SDD_LLD_BD_03]
- [STD_TC6_WB_TP09]::[SDD_LLD_API_02]
- [STD_TC6_WB_TP10]::[SDD_LLD_API_03]
- [STD_TC6_WB_TP11]::[SDD_LLD_API_04]
- [GUI_01]::support class for GUI result passing, no direct HLD mapping
- [GUI_02]::support class for menu buttons, no direct HLD mapping
- [GUI_03]::[SDD_HLD_GUI_01]
- [GUI_04]::[SDD_HLD_GUI_05]
- [GUI_05]::[SDD_HLD_GUI_02], [SDD_HLD_GUI_03], [SDD_HLD_GUI_04], [SDD_HLD_GUI_05]
- [GUI_06]::[SDD_HLD_GUI_01]
- [GUI_07]::[SDD_HLD_GUI_01]
- [GUI_08]::[SDD_HLD_GUI_01]
- [GUI_09]::[SDD_HLD_GUI_05]
- [GUI_10]::[SDD_HLD_GUI_01], [SDD_HLD_GUI_02], [SDD_HLD_GUI_03]
- [GUI_11]::support class for promotions, no direct HLD mapping
- [GUI_12]::[SDD_HLD_GUI_04], [SDD_HLD_GUI_05]
- [GUI_13]::[SDD_HLD_GUI_02], [SDD_HLD_GUI_03]
- [GM_01]::Classes in GM to function everything, no direct HLD mappinfg
- [GM_02]::[SDD_LLD_GM_01], [SDD_LLD_GM_02], [SDD_HLD_GM_02], [SDD_HLD_GM_01]
- [GM_03]::[SDD_LLD_GM_03], [SDD_LLD_GM_04]
- [GM_04]::[SDD_LLD_GM_05], [SDD_HLD_GM_05], [SDD_HLD_GM_04]
- [GM_05]::[SDD_LLD_BD_03]
- [GM_06]::[SDD_LLD_GM_06]
- [GM_07]::[SDD_HLD_GM_06]

Appendix

A – FEN Notation Explanation

Forsyth–Edwards Notation (FEN) is a standardized method used to represent a specific position in a chess game using a single line of text. Instead of storing the board as a graphical diagram or a long list of moves, FEN provides a compact way to encode all of the information necessary to reconstruct the exact state of a game at a particular moment. This format is widely used in chess software, databases, and engines because it allows positions to be easily saved, shared, and analyzed by computer programs. By reading a FEN string, a chess engine or application can immediately recreate the board and determine what moves are legal from that position.

A FEN string consists of six fields separated by spaces, each containing different information about the game. The first field describes the piece placement on the board. The board is written rank by rank starting from rank 8 (Black's back rank) down to rank 1 (White's back rank). Within each rank, pieces are represented by letters: uppercase letters represent White's pieces and lowercase letters represent Black's pieces. The letters correspond to the piece names (`K`/`k` for king, `Q`/`q` for queen, `R`/`r` for rook, `B`/`b` for bishop, `N`/`n` for knight, and `P`/`p` for pawn). Empty squares are represented by numbers indicating how many empty squares occur consecutively in that rank. Slashes (`/`) are used to separate the ranks.

The second field indicates which player has the move, using `w` for White and `b` for Black. The third field records castling availability, showing which sides still retain the right to castle. This is represented by the letters `K` and `Q` for White's kingside and queenside castling rights, and `k` and `q` for Black's. If neither side can castle, a dash (`-`) is used instead. The fourth field represents the en passant target square, which is the square where an en passant capture would be possible on the next move if one exists. If there is no en passant opportunity, this field is also written as `-`.

The final two fields provide information related to move counting. The halfmove clock records the number of halfmoves (individual player moves) since the last pawn move or capture, which is used to enforce the fifty-move rule. The fullmove number counts the number of complete moves in the game and increases after Black's move. Together, these fields ensure that the game state can be resumed accurately from the position.

For example, the starting position of a chess game is represented by the FEN string:

```
rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
```

This notation indicates the initial arrangement of pieces, that it is White's turn to move, that both sides still retain all castling rights, that there is no en passant target square, and that the game is at the beginning with zero halfmoves played and the first full move about to begin. Because FEN contains all the information necessary to reconstruct the board and game state, it serves as a fundamental tool in chess programming and analysis. Chess engines, databases, and graphical interfaces frequently rely on FEN to load positions quickly, test algorithms, and store positions without needing to record the entire history of moves that produced them.